CS543 · Web Services · Securing the API

API Security Tutorial

Part 0 · Install the two tools

Start from your Catalogue folder (the one with `server.js`). Install two packages alongside Express:

```
Terminal
$ npm install jsonwebtoken    # create & verify JWTs
$ npm install bcryptjs        # hash & check passwords
```

- `jsonwebtoken` — `jwt.sign()` makes a signed token; `jwt.verify()` checks one.
- `bcryptjs` — `bcrypt.hashSync()` turns a password into a one-way hash; `bcrypt.compareSync()` checks a guess against it.

Part 1 · Store users with hashed passwords

A login needs users to check against. **Never store a password as plain text** — store a bcrypt hash. For the demo we seed two users at startup; a real app keeps them in the database (Lecture 10). Open `server.js` and add this near the top:

```
server.js
const express = require('express');
const jwt     = require('jsonwebtoken');
const bcrypt  = require('bcryptjs');
const app = express();
app.use(express.json());

const SECRET = 'dev-secret-change-me';   // real life: process.env.JWT_SECRET

// two users -- passwords hashed once at startup, never stored in plain text
const users = [
  { id: 1, email: 'asha@campus.edu',    role: 'customer',   hash: bcrypt.hashSync('customer123', 10) },
  { id: 2, email: 'mealhub@campus.edu', role: 'restaurant', hash: bcrypt.hashSync('resto123', 10)    },
];
```

Why hash?
A hash is one-way. You can check a guess with `bcrypt.compareSync`, but you can never read the password back — so a stolen database reveals nothing usable. You never "decrypt" a password.

Part 2 · The login route — password in, token out

When a user logs in, verify the password, then **sign a JWT** that carries who they are and their role.

```
server.js
// POST /login -- verify the password, hand back a JWT
app.post('/login', (req, res) => {
  const user = users.find(u => u.email === req.body.email);
  if (!user || !bcrypt.compareSync(req.body.password, user.hash))
    return res.status(401).json({ error: 'bad login' });
  const token = jwt.sign({ sub: user.id, role: user.role }, SECRET, { expiresIn: '1h' });
  res.json({ token });
});
```

`jwt.sign` packs the payload (`sub` = the user, `role` = their role) plus an expiry, seals it with `SECRET`, and returns one string shaped `header.payload.signature`. The client stores it and sends it back on every request.

Part 3 · The auth middleware — who are you?

A **middleware** runs before the protected routes. It reads the token from the `Authorization` header, verifies the signature, and either attaches the user and continues, or replies `401`.

```
server.js
// authentication -- verify the token (who are you?)
```

```
function auth(req, res, next) {
  const token = (req.get('Authorization') || '').split(' ')[1];   // "Bearer <token>"
  try { req.user = jwt.verify(token, SECRET); next(); }           // ok -> carry on
  catch { res.status(401).json({ error: 'invalid or missing token' }); }
}

// GET /items -- any authenticated user may read the menu
app.get('/items', auth, (req, res) => res.json(items));
```

`jwt.verify` recomputes the signature with `SECRET` and checks the expiry. If anything is wrong — missing, tampered, or expired — it throws, and we reply `401`. Putting `auth` before the handler means the handler only runs for a valid token.

Part 4 · Try the authentication

(Add the `items` array and `app.listen(3000, ...)` from the previous tutorial.) Start the server, then try reading the menu with no token, log in, and try again with the token:

```
Terminal
$ node server.js
Secured Catalogue on http://localhost:3000
```

```
authentication  ·  who are you?

# 1  read the menu with no token  —  the server doesn't know you
$ curl -i localhost:3000/items
HTTP/1.1 401 Unauthorized
{"error":"invalid or missing token"}

# 2  log in as the restaurant  —  get a JWT back
$ curl -X POST localhost:3000/login \
      -d '{"email":"mealhub@campus.edu","password":"resto123"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjIs..."}

# 3  read the menu WITH the token  —  now it works
$ curl -i localhost:3000/items -H "Authorization: Bearer $TOKEN"
HTTP/1.1 200 OK
[{"id":1,"name":"Veg Thali","price":90}, ... ]
```

No token → 401. Log in → a JWT. Send it as Authorization: Bearer <token> → 200 and the menu.

> **Checkpoint**
> You have done **authentication** — the server now refuses anyone it can't identify, and accepts a valid token. `401` means “I don't know who you are.”

Part 5 · Authorization — are you allowed?

Authentication told us **who**. Authorization decides **what they may do**. A customer may read the menu, but only a restaurant may add a dish. The role already travelled inside the token (we put it there at login), so we just check it with a second middleware:

```
server.js
// authorization -- require one of the allowed roles
function requireRole(...allowed) {
  return (req, res, next) =>
    allowed.includes(req.user.role) ? next() : res.status(403).json({ error: 'forbidden' });
}

// POST /items -- only a restaurant (or admin) may add a dish
app.post('/items', auth, requireRole('restaurant', 'admin'), (req, res) => {
  const { name, price } = req.body;
  if (!name) return res.status(422).json({ error: 'name required' });
  const item = { id: nextId++, name, price: Number(price) || 0 };
  items.push(item);
  res.status(201).json(item);
```

```
});
```

Three guards run in order: `auth` (who are you → 401), `requireRole` (may you → 403), then the handler (do it → 201). The role is safe to trust because it sits inside the **signed** token — a customer can't edit it to "restaurant" without breaking the signature.

Part 6 · Try the authorization

```
authorization  ·  are you allowed?

# 4  the RESTAURANT adds a dish      (token role: restaurant)
$ curl -X POST localhost:3000/items -H "Authorization: Bearer $RESTO" \
      -d '{"name":"Masala Dosa","price":60}'
HTTP/1.1 201 Created
{"id":4,"name":"Masala Dosa","price":60}

# 5  a CUSTOMER tries the same       (token role: customer)
$ curl -X POST localhost:3000/items -H "Authorization: Bearer $CUST" \
      -d '{"name":"Hack Burger","price":99}'
HTTP/1.1 403 Forbidden     ← authenticated, but not allowed
{"error":"forbidden"}
```

The restaurant's token → 201, dish added. The customer's token → 403: authenticated, but not allowed.

401 vs 403 — don't mix them up

`401 Unauthorized` = I don't know who you are (no / bad token). `403 Forbidden` = I know you, but you can't do this. This is the most common API mistake.

Part 7 · Understanding the lock

- **Why the token can't be forged** — the signature is computed from the payload + `SECRET`. Change the payload (e.g. `role` → `admin`) and the signature no longer matches; without `SECRET` an attacker can't produce a valid one.
- **Keep the secret out of code** — in real life read it from `process.env.JWT_SECRET` (a `.env` file, never committed). If it leaks, anyone can mint tokens.
- **Tokens expire** — `expiresIn: '1h'` limits the damage of a leaked token to an hour.
- **Always over HTTPS** — a token sent over plain HTTP is read by anyone on the network. In production, serve only over HTTPS.
- **Validation is separate** — the `422` name check stays. Security says *who / may you*; validation says *is the data clean*. Both gates matter.

Security checklist

Passwords hashed (bcrypt), never plaintext · secret in `.env`, token has an expiry · authenticate (401) before authorize (403) · check the specific resource too (can you edit *this* order? — the lecture's IDOR point) · HTTPS in production.

Recap — what you built

- A `POST /login` that checks a password with **bcrypt** and returns a signed **JWT**.
- An `auth` middleware — **authentication**: verify the token, or `401`.
- A `requireRole` middleware — **authorization**: check the role, or `403`.
- A secured Catalogue: `GET /items` for any signed-in user, `POST /items` for a restaurant only.

Your final secured server.js:

server.js — complete

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const app = express();
app.use(express.json());
const SECRET = 'dev-secret-change-me';


const users = [
  { id:1, email:'asha@campus.edu',    role:'customer',   hash:bcrypt.hashSync('customer123',10) },
  { id:2, email:'mealhub@campus.edu', role:'restaurant', hash:bcrypt.hashSync('resto123',10)    },
];
let items = [ {id:1,name:'Veg Thali',price:90}, {id:2,name:'Paneer Roll',price:70} ];
let nextId = 3;


app.post('/login', (req, res) => {
  const user = users.find(u => u.email === req.body.email);
  if (!user || !bcrypt.compareSync(req.body.password, user.hash))
    return res.status(401).json({ error: 'bad login' });
  const token = jwt.sign({ sub:user.id, role:user.role }, SECRET, { expiresIn:'1h' });
  res.json({ token });
});


function auth(req, res, next) {
  const token = (req.get('Authorization') || '').split(' ')[1];
  try { req.user = jwt.verify(token, SECRET); next(); }
  catch { res.status(401).json({ error:'invalid or missing token' }); }
}
function requireRole(...allowed) {
  return (req, res, next) =>
    allowed.includes(req.user.role) ? next() : res.status(403).json({ error:'forbidden' });
}


app.get('/items', auth, (req, res) => res.json(items));
app.post('/items', auth, requireRole('restaurant','admin'), (req, res) => {
  const { name, price } = req.body;
  if (!name) return res.status(422).json({ error:'name required' });
  const item = { id: nextId++, name, price: Number(price)||0 };
  items.push(item); res.status(201).json(item);
});


app.listen(3000, () => console.log('Secured Catalogue on http://localhost:3000'));
```

Where to go next

Add an `admin` role that may do everything; add an **object-level** check (a customer may read only *their own* orders); then move users and the secret into a database and a `.env` file. The lab assignment has you secure your own CampusEats service exactly this way.